SIS3204Y(5) Natural Language Processing

Week 3 Revision Paper: Feature Extraction & Text Encoding (with Answer Key)
University of Mauritius | MSc AI Level 2

Answer ALL 15 questions. Each answer is printed directly under its question; marks for each point are shown in grey brackets [ ].

Q1. Define feature extraction in NLP and state THREE reasons why it is important. [4 marks]

Answer

- **Feature extraction** is the process of converting raw text into a set of meaningful **numerical features (vectors)** that a machine learning model can understand and use. [1]
- **Computers only work with numbers**, so text must be converted before processing. [1]
- Features capture the **most useful information** from the text, which **improves model performance**. [1]
- It **reduces complexity and noise** in the data (also makes learning more efficient and enables generalisation). [1]

Q2. Explain FOUR reasons why words cannot be fed directly to a machine learning model. [4 marks]

Answer

- **Models require numbers:** operations such as addition, multiplication and dot products are not defined on strings (e.g. "cat" + "dog" is invalid, 0.25 + 0.75 is valid). [1]
- **Words have no natural numerical meaning:** they are symbols with no inherent magnitude or order; "cat" is not numerically closer to "dog" than to "car". [1]
- **Dimensionality mismatch:** a vocabulary can have thousands or millions of words, but models expect fixed, much smaller input vectors (e.g. 10,000 words vs. vectors of size 128). [1]
- **Lack of generalisation:** a word unseen in training (e.g. "hamster") cannot be reasoned about at inference time. [1]

Q3. Describe the text representation pipeline that converts raw text into numerical feature vectors. Use the sentence "The cat sat on the mat." as your example. [6 marks]

Answer

- **1. Raw text:** original text as collected, e.g. "The cat sat on the mat." [1]
- **2. Preprocessing:** lowercasing, removing punctuation, stopwords and extra spaces, optional stemming/lemmatisation. Result: "the cat sat on the mat". [1]
- **3. Tokenisation:** split into tokens: [the, cat, sat, on, the, mat]. [1]
- **4. Vocabulary building:** list of unique tokens, size |V| = 5: 0:the, 1:cat, 2:sat, 3:on, 4:mat. [1]
- **5. Indexing (mapping):** replace each token with its integer index, giving the sequence [0, 1, 2, 3, 0, 4]. [1]
- **6. Encoding** (one-hot, count vectors, etc.) and **7. Numerical feature vectors** (dense or sparse) ready as input to ML models. [1]

Q4. Define a vocabulary. Given the corpus: Doc 1 = "the cat sat on the mat" and Doc 2 = "the dog sat on the log", build the vocabulary (assign indices from 0 in order of first appearance) and integer-encode Doc 2. [5 marks]

Answer

- A **vocabulary** is the list of all **unique tokens (words)** in the corpus; each token is assigned a unique index used later for encoding. [1]
- Vocabulary (|V| = 7): **the=0, cat=1, sat=2, on=3, mat=4, dog=5, log=6**. [2]
- Doc 2 tokens: [the, dog, sat, on, the, log]. [1]
- Integer-encoded Doc 2 = **[0, 5, 2, 3, 0, 6]** (both occurrences of "the" share index 0). [1]

Q5. Explain integer (label) encoding. Discuss FOUR problems associated with it, giving an example for each. [6 marks]

Answer

- **Integer encoding** assigns each unique token a unique integer (usually starting at 0) and replaces tokens with their integers. It is simple, fast and memory efficient. [2]
- **False order:** numbers create an artificial ranking; e.g. dog=2, cat=5, tiger=9 makes the model think tiger is "greater than" dog. [1]
- **False distance:** |5 - 2| = 3 and |5 - 9| = 4 suggest cat is closer to dog than to tiger, but the gaps mean nothing for words. [1]
- **No semantic generalisation:** IDs carry no meaning; given king=1, queen=2, the ID 3 for "prince" gives no clue about its relation. [1]
- **Sensitive to vocabulary changes:** adding a word (e.g. "apple" at the start) shifts IDs (dog 2 to 3, cat 5 to 6), breaking consistency; also unsuitable for neural models alone (use embeddings). [1]

Q6. Using the vocabulary [the, cat, sat, on, mat, again] (V = 6), one-hot encode the sentence "the cat sat on the mat again". State the shape of the resulting matrix and the one-hot vector for "sat". [5 marks]

Answer

- Positions: the=1, cat=2, sat=3, on=4, mat=5, again=6. [1]
- Sentence index sequence: [1, 2, 3, 4, 1, 5, 6]. Each word is replaced by a binary vector with a single 1 at its position (both "the" use the same vector). [1]
- Resulting matrix shape = **V x N = 6 x 7** (rows = vocabulary words, columns = words in the sentence). [2]
- Vector for "sat" = **[0, 0, 1, 0, 0, 0]** (length 6). [1]

Q7. Discuss THREE advantages and THREE limitations of one-hot encoding. [6 marks]

Answer

- **Advantage 1:** removes false ordinal relationships and distances (no ranking between words). [1]
- **Advantage 2:** treats all categories equally, with no bias towards larger or smaller integers. [1]
- **Advantage 3:** simple, safe and widely compatible with linear models, logistic regression and neural networks; each column is interpretable. [1]
- **Limitation 1: high dimensionality / sparsity**: as many columns as categories (10,000 categories gives 10,000 columns, mostly 0). [1]
- **Limitation 2: ignores relationships**: treats cat and tiger as unrelated, cannot capture similarity or hierarchy. Not scalable to high-cardinality features (e.g. millions of user IDs). [1]
- **Limitation 3: higher computational cost** (more features means longer training and prediction); often unnecessary for tree-based models where integer encoding suffices. [1]

Q8. What is the sparse vector problem? A model uses a vocabulary of 100,000 words. Calculate the number of entries needed to store 1 million one-hot vectors and the percentage that are zeros. [4 marks]

Answer

- The **sparse vector problem**: one-hot vectors are mostly 0s with a single 1, so for large vocabularies they become huge and mostly uninformative. [1]
- Each vector has length 100,000, so 1,000,000 vectors need 1,000,000 x 100,000 = **100 billion entries**. [1]
- Only 1 million entries are 1s, so **99.999%** are zeros. [1]
- Consequences: high memory usage, wasted computation on zeros, inefficient storage, not scalable. [1]

Q9. Explain why one-hot vectors do not capture meaning. [4 marks]

Answer

- They represent only **presence (1) or absence (0)**: identity, not meaning. [1]
- All vectors are **orthogonal** (independent), so every pair of words is equally distant; "cat" is as unrelated to "tiger" as to "car". [1]
- They cannot represent **synonyms** (big and large), **hierarchies** (animal, mammal, cat) or **contextual similarity** (bank in "river bank" vs. "money bank"), and cannot generalise knowledge to related categories. [1]

- They carry **no information about frequency or intensity**. Meaning needs dense representations such as word embeddings. [1]

Q10. Compare sparse and dense representations using a table with SIX aspects. [6 marks]

Answer

Aspect	Sparse (One-Hot, BoW, TF-IDF)	Dense (Word2Vec, GloVe, FastText, BERT)
Vector size	Very high (10K to 1M+, = V)	Low (d, typically 50 to 500)
Non-zero values	Very few (mostly 0s)	All (or almost all) non-zero
Data type	Binary (0/1) or counts / TF-IDF	Real-valued numbers
Captures meaning	No	Yes
Captures similarity	No (all words equally distant)	Yes (similar words are close)
Memory / computation	High memory, slower	Low memory, faster

1 mark per correctly contrasted aspect (6 marks). Key idea: sparse vectors tell us which words are present; dense vectors tell us what words mean and how they relate.

Q11. Explain why word embeddings are needed. In your answer, state what traditional encodings (One-Hot, Bag-of-Words, TF-IDF) miss and FOUR things word embeddings provide. [6 marks]

Answer

- **One-Hot** captures identity only: no meaning, all words equally distant, sparse, poor generalisation. [1]
- **Bag-of-Words / Count** captures presence/frequency but ignores word order and context, has no semantic similarity, and is high-dimensional and sparse. [1]
- **TF-IDF** captures importance of words in a document but still has no meaning or relationships and remains high-dimensional and sparse. [1]
- Word embeddings represent words as **dense vectors of real numbers in a continuous space**. They provide: [1]
- **Meaning** (similar words have similar vectors) and **relationships** (e.g. king - man + woman is approx. queen). [1]
- **Compact, dense vectors** (50 to 300 dimensions), **better generalisation** to unseen words/contexts, and improved downstream NLP performance (classification, QA, MT, summarisation, sentiment). [1]

Q12. What is Word2Vec? State the distributional hypothesis and explain how Word2Vec learns word representations. [6 marks]

Answer

- **Word2Vec** is a family of **unsupervised** learning models that learn dense, low-dimensional vector embeddings of words from large text by capturing the **context** in which words appear (introduced by Tomas Mikolov et al., Google, 2013). [1]
- **Distributional hypothesis:** words that occur in **similar contexts** tend to have **similar meanings**. [1]
- **Learning process:** take a large corpus, preprocess it (tokenise, lowercase), then slide a **context window** (size c) over sentences to generate (target, context) pairs. [1]
- **Objective:** predict context words from a target word (Skip-Gram) or the target from context words (CBOW), maximising the probability of the correct word predictions (optimised with stochastic gradient descent). [1]
- **Result:** each word gets a d-dimensional dense vector (typically 50 to 300), with similar words close together; no manual labelling is required. [1]
- Captures relationships such as king - man + woman is approx. queen and Paris - France + Italy is approx. Rome. [1]

Q13. (a) Compare the two Word2Vec architectures, CBOW and Skip-Gram, in a table covering SIX aspects. [6]
(b) In CBOW with window size 2, the context words for target "sat" are "the" and "cat". Given v_{the} = [0.22, -0.10, 0.41, 0.05] and v_{cat} = [0.31, 0.20, -0.13, 0.07], compute the context vector. [4]

[10 marks]

Answer

Aspect	CBOW (Continuous Bag of Words)	Skip-Gram
Direction	Context to Target	Target to Context
Input	Multiple context words	Single target (centre) word
Output	Single target word	Multiple context words
Best for	Frequent words, general meaning	Rare words, specific meanings
Training speed	Faster	Slower
Data efficiency	Works well with less data; small/medium datasets	Often needs more data; large datasets

(a) 1 mark per correctly contrasted aspect. Example for "the cat sat on the mat": CBOW learns [the, cat, on, the] to sat; Skip-Gram learns sat to [the, cat, on, the].

- **(b)** CBOW **averages** the context word vectors: $v_{context} = (v_{the} + v_{cat}) / 2$. [1]
- Dimension 1: (0.22 + 0.31)/2 = 0.265; Dimension 2: (-0.10 + 0.20)/2 = 0.050. [1]
- Dimension 3: (0.41 - 0.13)/2 = 0.140; Dimension 4: (0.05 + 0.07)/2 = 0.060. [1]
- $\mathbf{v_{context} = [0.265, 0.050, 0.140, 0.060]}$. This averaged vector is passed to an output (softmax) layer, which should give the highest probability to the target "sat". [1]

Q14. Discuss SIX limitations of Word2Vec and state how modern models address them. [6 marks]

Answer

- **Local context only:** a small window misses long-range dependencies (e.g. "cat" ... "house" in a long sentence). [0.5]
- **Ignores word order in CBOW:** "The dog chased the cat" and "The cat chased the dog" give the same context bag for "chased". [0.5]
- **No handling of polysemy:** "bank" (financial) and "bank" (river side) share one vector. [0.5]
- **Out-of-vocabulary (OOV) words:** unseen words (e.g. "giraffe") have no vector. [0.5]
- **Static embeddings:** same vector regardless of context ("apple" the fruit vs. the company). [0.5]
- **Limited to linear relationships** (king - man + woman is approx. queen) and struggles with complex, non-linear ones. [0.5]
- **Modern fixes (3 marks):** larger context and word order/position via **Transformers/attention**; **contextual embeddings** (ELMo, BERT, GPT); **subword tokenisation** (BPE, WordPiece) for OOV; deep models for non-linear relations. [3]

Marks: 0.5 per limitation (3 marks) + 3 marks for how modern models address them.

Q15. Trace the evolution of text representation from integer encoding to Word2Vec. Show how each technique tries to fix the weakness of the previous one and why Word2Vec is a step forward. [12 marks]

Answer

- **Starting point:** computers understand only numbers, so text must go through preprocessing, tokenisation, vocabulary building, indexing and encoding to give numerical features. [1]
- **Integer / label encoding:** each token gets a unique ID. Simple, memory efficient, good baseline, works with tree-based models. [1]
- **Weakness:** implies a **false order and distance** (e.g. zebra=4 > apple=0), misleading linear and distance-based models (linear regression, k-NN), no semantics, sensitive to vocabulary changes. [2]
- **One-hot encoding** fixes this: binary vector per word (1 at its index, 0 elsewhere) so there is no order, no distance and all words are treated equally; works well with linear models and neural networks. [2]
- **Weakness: sparse, high-dimensional** vectors (100,000-word vocabulary gives 100,000-length vectors, 99.999% zeros), high memory and computation, not scalable; all words are equidistant so **no meaning or similarity**; no frequency information. [2]
- **Count-based document representations** (Bag-of-Words, TF-IDF) give fixed-length document vectors capturing frequency/importance, but remain sparse, ignore word order and still carry no semantic meaning.

[1]

- **Word embeddings** fix this: **dense, low-dimensional** (50 to 300) real-valued vectors in a continuous space where similar words are close (cat, tiger, dog cluster; car, bus, truck cluster) and relationships emerge (king - man + woman is approx. queen). [1]
- **Word2Vec** learns these embeddings **automatically** from raw text using the **distributional hypothesis**, via **CBOW** (context to target) or **Skip-Gram** (target to context). Result: lower memory, faster computation, better generalisation and better downstream NLP performance. [1]
- **Remaining gap:** Word2Vec is static, local and cannot handle polysemy or OOV words, motivating contextual embeddings (ELMo, BERT, GPT). [1]

Summary chain: Text, then Integer IDs, then One-Hot (sparse, no meaning), then Word2Vec (dense, with meaning), then Contextual Embeddings.

Total: 90 marks (15 questions)